Quantx: An NSE Equity Paper-Trading Workspace with Boundary-Enforced Market Data, Depth-Based Local Order Matching, and Transactional SafePoint Recovery

Bhavik Achotani
Department of Information Technology
Thadomal Shahani Engineering College, University of Mumbai
Mumbai, India
achotanibhavikoffice@gmail.com

Abstract—In this work, we developed Quantx, an exchange-style paper-trading workspace for National Stock Exchange of India (NSE) equities in which simulated orders are matched locally against streamed market depth and protected by transactional state snapshots. The system is organized as four tiers: a React client, a FastAPI service tier, an in-process market-data decoding and matching tier, and a PostgreSQL ledger paired with a Redis quote cache. Its foundational design principle is a strict architectural boundary between a market-data plane, which may only read authentication, instrument, and quote data from upstream providers, and an accounting plane, which alone creates orders, fills, statutory charges, cash movements, positions, and recovery snapshots; paper orders are never routed to an external broker or exchange order interface. Inside this boundary, the system normalizes prices to tick size, validates circuit, board-lot, and freeze constraints, walks five-level bid and ask depth with partial fills, applies product-aware brokerage and Indian statutory charges, discards duplicate and older-timestamp packets, refuses market orders when depth is stale, and provides atomic SafePoint state rollback alongside non-destructive multi-modal What-If stress testing and a guardrailed non-advisory learning guide. We evaluated the integrated workflow against a last-price, zero-charge, arrival-order baseline (B0) across $N = 50$ scripted orders spanning Typical ($N_1 = 20$), Boundary ($N_2 = 20$), and Adversarial ($N_3 = 10$ valid orders plus $N_{\text{adv}} = 50$ injected fault events) scenario profiles on a live PostgreSQL 16.15 and Redis 7.4.11 stack. Quantx achieved zero ledger-reconciliation error (mean = $0.00\,INR$, max = $0.00\,INR$ vs. B0 mean = $19{,}684.41\,INR$, max = $227{,}398.32\,INR$), zero fill-price deviation ($0.00\,INR/share$ vs. B0 $1.60\,INR/share$ across $N_f = 73$ fills), zero charge deviation ($0.00\,INR/fill$ vs. B0 $33.08\,INR/fill$), and zero adversarial fault leaks across all 50 injected disorder, staleness, idempotency, and rule-violation events, while maintaining sub-millisecond quote ingestion (p99 = $0.06\,ms$) and sub-120 ms five-level match-and-commit latency (p50 = $71.02\,ms$, p99 = $118.02\,ms$). These results support the engineering feasibility of boundary-enforced local paper execution for NSE equity practice; they do not establish fidelity to live exchange queue priority, educational learning outcomes, or production brokerage readiness.

Index Terms—Paper Trading, Order Matching, Limit Order Book, Market Depth, Indian Equities, Transaction Charges, Idempotent Submission, Out-of-Order Market Data, SafePoint State Recovery, Financial Education Software.

I. Introduction

Software that enables a learner to practice securities trading must reconcile two classes of information whose ground truth is owned by distinct parties. The first class is market data: instrument reference masters, top-of-book quotes, and multi-level order-book depth that originate at an exchange or its licensed vendor, arrive as an asynchronous, high-rate stream, and cannot be altered by the client application [1]–[3]. The second class is account state: orders, partial fills, statutory charges, cash balances, positions, and recovery checkpoints that exist solely because the application records them, and whose internal consistency the application must guarantee through atomic transactional storage [4], [5]. A practice environment is trustworthy only when the first class of information is treated strictly as read-only input and the second as an authoritative, auditable ledger.

Unconstrained retail trading simulators typically blur this separation and exhibit three recurring failure modes:

- Fill-Model Fragility: Filling an order of arbitrary size at the last traded price ignores finite displayed liquidity at each price level and the bid–ask spread, even though empirical execution prices depend directly on how far an order walks the limit order book and on order quantity [6]–[9].
- Charge and Product-Rule Blindness: Transaction costs and portfolio turnover are primary determinants of individual investors' net returns [10]–[12]. In Indian cash equities, statutory charges—Securities Transaction Tax (STT), exchange transaction fees, SEBI turnover fees, stamp duty, and Goods and Services Tax (GST)—differ sharply by product type (Cash and Carry delivery, CNC, versus Margin Intraday Square-off, MIS) and by order side. A simulator that omits per-fill statutory charges or exchange lot/circuit rules reports systematically inflated profit and loss.
- Feed-Disorder and State-Corruption Fragility: Real

network feeds deliver duplicated, delayed, and out-of-order frames in which processing-time arrival order diverges from exchange event-time order [13]–[15]. A simulator that applies packets in arrival order allows a delayed quote to overwrite a newer book state, and a simulator without transactional state snapshots or non-advisory guardrails leaves learners unable to recover cleanly from accidental over-leveraging or simulated flash-crash exercises.

This study evaluates an integrated, boundary-enforced workspace—Quantx—rather than any single mechanism in isolation. The workflow combines server-side-only provider credentials; an architectural provider boundary that excludes order-placement endpoints; big-endian binary frame decoding with paise-to-rupee normalization; event-time ordering and duplicate suppression; a configurable stale-depth guard for market orders; tick rounding and circuit, board-lot, freeze, product, and buying-power validation at admission; conservative cash reservation; idempotent order submission; five-level depth-walking matching with partial fills and Immediate-or-Cancel (IOC) remainder cancellation; product-aware per-fill statutory charges; single-transaction persistence of orders, trades, charge breakdowns, cash-ledger entries, and positions; an immutable order-event audit trail; account-level Market Stress Mode; a transactional SafePoint Recovery Shield with autonomous fault rollback; a multi-modal What-If scenario laboratory; and a dual-mode, SEBI-compliant non-advisory learning guide.

A. Problem Statement

Existing literature treats these concerns in separate silos: market-microstructure models characterize limit order book dynamics [3], [9], empirical finance studies quantify retail transaction drag [12], [16], and distributed-systems research formalizes out-of-order stream processing, idempotency, and ACID transactions [4], [13], [17]. Applied piecemeal, each leaves critical vulnerabilities at the application boundary. A depth-walking fill model without an integrated statutory charge engine produces drifted cash balances; a charge engine without a stale-depth gate bills executions against frozen market snapshots; and an accounting ledger without an enforced provider boundary risks leaking practice orders to a live brokerage endpoint or leaving learner portfolios wedged after stress experiments.

The research and systems-engineering problem is to design and verify a unified execution architecture for NSE equity practice in which (i) upstream market data is strictly read-only input across a zero-order provider boundary, (ii) every order, partial fill, statutory charge component, cash movement, position update, and state checkpoint is computed and persisted atomically inside the application, and (iii) the ledger remains exact to the paisa under duplicate, out-of-order, bursty, and stale market updates, while explicitly delineating which properties are empirically measured and which remain outside the scope of simulation.

B. Project Objectives

- Implement a local paper-trading execution engine supporting MARKET, LIMIT, Stop-Loss Limit (SL), and Stop-Loss Market (SL-M) orders with five-level partial fills, modification, cancellation, and DAY/IOC validity.
- Enforce an authenticated market-data plane behind a strict provider boundary that permits authentication, instrument-master synchronization, and quote streaming only.
- Construct an ACID transactional accounting plane that persists per-level fills, six-component Indian statutory charges, double-tagged cash-ledger entries, signed positions, and atomic SafePoint state snapshots.
- Develop deterministic feed-disorder filtering, stale-depth market-order gating, non-destructive multi-modal What-If stress analysis, and guardrailed educational assistance.
- Evaluate the integrated system against a naive baseline (B0) across Typical, Boundary, and Adversarial profiles on a live PostgreSQL and Redis deployment, reporting full percentile distributions alongside exact reconciliation audits.

C. Contributions

1) A four-tier system architecture enforcing a hard separation between the read-only market-data plane and the transactional accounting and resilience plane (Sections III and IV).
2) A formal mathematical specification of five-level depth-walking order matching, conservative buying-power reservation, product-aware Indian statutory charges, signed position accounting, SafePoint state diffing and rollback, and multi-modal What-If shock projections, with every variable and unit explicitly defined (Section V).
3) A four-phase execution workflow, module configuration registry, validated request contract, and dual-mode guardrailed support architecture (Section VI).
4) A reproducible empirical evaluation across $N = 50$ scripted orders ($N_f = 73$ fills) and $N_{\text{adv}} = 50$ adversarial fault injections comparing Quantx against baseline B0, paired with candid validity boundaries and four pre-registered future evaluation protocols (Sections VII–IX).

The remainder of this paper is structured as follows. Section II reviews related literature and formulates three falsifiable hypotheses. Section III presents the four-tier architecture. Sections IV and V detail the market-data plane and the matching, charging, recovery, and scenario mechanisms. Section VI documents runtime configuration,

structured validation, and the guardrailed assistant. Section VII reports the experimental setup and empirical results. Section VIII discusses architectural limitations, code–documentation discrepancies, and ethical boundaries, and Section IX concludes with reproducible future protocols.

II. Related Work

A. Market Microstructure and Limit Order Book Execution

Theoretical and empirical market-microstructure research establishes why execution prices depend on displayed order-book liquidity rather than the most recent trade print. Comprehensive surveys of limit order books characterize spread dynamics, depth profiles, and queue priority [3], while stochastic queueing models demonstrate how top-of-book and multi-level depth govern short-horizon fill probabilities [9]. Optimal execution frameworks link order size and execution horizon to temporary and permanent market impact [6], equilibrium models connect adverse selection and order flow to bid–ask spreads [7], [8], and empirical studies document how markets digest multi-level liquidity consumption [1], [2], [18]. While these works formalize exchange matching and price formation, they do not specify an application-level software architecture that reproduces multi-level depth walking and partial-fill accounting inside an isolated learner workspace while guaranteeing that external market feeds remain strictly read-only.

B. Transaction Costs and Retail Investor Performance

Empirical studies of individual brokerage accounts consistently show that transaction costs and excessive turnover significantly erode net equity returns [10], [11], producing substantial aggregate wealth transfers from active retail traders [12]. In the Indian market context, a landmark study by the Department of Economic and Policy Analysis (DEPA) of the Securities and Exchange Board of India (SEBI) documented that 89% of individual traders in the equity futures and options (F&O) segment incurred net losses, with transaction costs consuming a major share of gross trading turnover [16]. Although SEBI's empirical report focused on derivatives rather than the cash-equity segment studied here, its findings underscore the pedagogical necessity of modeling product-specific brokerage, STT, stamp duty, exchange turnover fees, SEBI regulatory fees, and GST on every simulated fill rather than reporting frictionless gross returns.

C. Experiential Learning, Simulation Fidelity, and Overfitting Risks

Experiential learning theory emphasizes concrete action and reflective observation in risk-free environments prior to live performance [19]. In quantitative finance, however, simulated performance carries severe epistemic hazards: Bailey et al. [20] prove that repeated strategy testing on historical or synthetic paths easily generates false discoveries through backtest overfitting. Computational reproducibility standards therefore require that simulation claims be accompanied by exact parameter disclosures, deterministic replay artifacts, and explicit scope boundaries [21], [22]. Because no controlled human-subject study was conducted in this work, we make zero claims regarding educational efficacy or live trading skill transfer, focusing strictly on systems-level accounting and execution conformance.

D. Event-Time Disorder, Idempotency, and Transactional Recovery

Distributed stream-processing literature distinguishes event time (when an exchange generates a quote) from processing time (when a downstream server receives it), demonstrating that network jitter and reconnection bursts inevitably cause out-of-order and duplicate arrivals [13]–[15]. Simultaneously, enterprise data-intensive systems rely on client-supplied idempotency keys to render network retries safe [17] and on ACID relational transactions with snapshot recovery to prevent partial state corruption [4], [5], [23]. Existing financial simulators rarely integrate event-time quote filtering, stale-depth market-order blocking, idempotent order admission, and user-invocable portfolio SafePoint rollbacks into a single auditable pipeline.

Research Gap and Hypotheses. To our knowledge, no prior peer-reviewed system combines, in one empirically audited workflow, (i) a zero-order provider boundary for NSE equities, (ii) five-level depth-walking partial fills with six-component Indian statutory charges, (iii) event-time disorder and staleness gates feeding an ACID ledger, and (iv) atomic SafePoint portfolio recovery. We formulate this gap as three falsifiable hypotheses tested in Section VII against a defined baseline B0 (which executes full quantities at the last traded price, applies zero charges, processes packets in arrival order without staleness checks, and omits idempotency and admission rule gates):

- H1 (Ledger and Fill Conformance): Across Typical, Boundary, and Adversarial order profiles, Quantx yields strictly smaller ledger-reconciliation error ($\varepsilon_M \leq 0.01\,INR$), fill-price deviation (δ_p), and charge deviation than B0, with zero fills violating limit-price bounds.
- H2 (Disorder and Staleness Resilience): Under adversarial streams containing out-of-order exchange timestamps and stale quotes ($A_s(t) > \theta$), Quantx records zero older-packet cache overwrites and zero market-order executions on stale depth, whereas B0 records at least one failure in each category.
- H3 (Idempotency and Rule-Gate Invariance): Under repeated submissions of an identical idempotency_key and injections of off-tick, out-of-circuit, lot-violating, freeze-exceeding, or underfunded requests, Quantx produces zero duplicate orders, fills, or ledger entries and accepts zero invalid orders.

III. Proposed System and Architecture

Quantx is structured as a four-tier web and data-processing architecture deployed via Docker Compose. External market-data, news, and optional language-model providers reside strictly outside the application boundary and interact only through read-only adapters in the application and market-data tiers.

- Tier 1 — Presentation Tier (React 18, TypeScript, Vite, Tailwind CSS): Renders the multi-pane trading workspace, interactive order ticket, five-level market-depth ladder, data-source provenance badges (021_live, indianapi, demo), What-If scenario laboratory, SafePoint Recovery Shield drawer, and Quantx Guide panel. It maintains an authenticated WebSocket connection (/api/market/ws) with exponential-backoff reconnection.
- Tier 2 — Application Tier (FastAPI 0.156, Pydantic 2.14, SQLAlchemy 2.1): Exposes RESTful resource controllers [24] and the quote WebSocket endpoint [25], enforces Pydantic request schemas, hashes user passwords using memory-hard Argon2id [26], and issues HMAC-SHA256 signed JWT session tokens [27] inside HttpOnly, SameSite=Lax cookies.
- Tier 3 — Market-Data, Matching, and Resilience Tier (In-Process Domain Services): Authenticates server-side to the upstream market feed, decodes binary frames (TC 1 and TC 3), enforces event-time ordering and duplicate suppression, executes the five-level depth-walking paper matcher and statutory fee calculator (PaperTradingEngine), runs the elected 15:20 IST intraday MIS square-off worker, executes multi-modal What-If stress simulations (ScenarioEngine), and manages SafePoint snapshots and autonomous fault rollbacks (RecoveryEngine).
- Tier 4 — Persistence Tier (PostgreSQL 16.15, Redis 7.4.11): PostgreSQL serves as the single authoritative ACID system of record for users, accounts, instruments, orders, trades, trade charges, order events, positions, cash-ledger entries, scenario runs, and portfolio SafePoints. Redis maintains low-latency latest-quote hashes (quantx:quote:<symbol>) and a capped append-only stream (quantx:market:quotes, MAXLEN 10,000).

Fig. 1 illustrates the four architectural tiers, the red dashed zero-order provider boundary, and the separation between the Market-Data Plane and the Accounting & Resilience Plane. Password recovery follows OWASP guidelines [28]: single-use reset tokens are persisted exclusively as SHA-256 digests, expire after 30 min, and are rate-limited to one request per minute per account.

A. Separation of Responsibilities

The architecture enforces an invariant separation between the market-data plane and the accounting and resilience plane. The market-data plane (MarketDataService, ProviderClient) may authenticate to an external provider, synchronize the NSE instrument catalog, and ingest binary quote and depth frames; it has no access to order-placement endpoints, holds no user account state, and never writes to the financial ledger. Conversely, the accounting and resilience plane (PaperTradingEngine, RecoveryEngine, ScenarioEngine) alone creates or mutates orders, trades, statutory charge records, cash-ledger entries, positions, and SafePoint snapshots. It never treats a depth-less REST snapshot (indianapi) as executable market depth, never coerces a missing quote to a zero price, and tags every quote with an explicit provenance label (021_live, indianapi, or demo) so the user interface always reflects data origin.

IV. Boundary-Enforced Market-Data Plane

The market-data plane transforms upstream frames—which arrive in big-endian binary format, express prices in integer paise, and are subject to network duplication, reordering, and disconnections—into normalized, event-time-ordered, rupee-denominated quotes suitable for deterministic consumption by the matching engine. Fig. 2 depicts the end-to-end ingestion, filtering, caching, and fan-out pipeline.

A. Provider Boundary and Source Provenance

The backend authenticates to the upstream market-data provider (021 Developer API) using server-held credentials to obtain a session token, synchronize the NSE cash equity instrument master CSV, and request a short-lived ephemeral key that opens the upstream WebSocket. Provider order-submission and order-update endpoints are excluded from the codebase. Every quote dictionary carries an explicit data_source field:

1) 021_live: Full five-level depth or last-traded-price packets streamed over the authenticated provider WebSocket.
2) indianapi: Periodic REST price snapshots containing last price, high, low, and volume but zero order-book depth. The paper engine ignores these snapshots in remember_quote and explicitly blocks MARKET orders against snapshot-only quotes (HTTP 409).
3) demo: Deterministic-structure synthetic five-level depth generated every 1.4 s per subscribed symbol when development mode is active, with per-level spread $\max\{0.025\%$ of last price, $0.01\,INR\}$, displayed quantities drawn uniformly from [100, 2,200] shares, and a bounded random-walk step within $\pm 0.09\%$. Fresh 021_live quotes automatically preempt demo ticks for the same symbol, and demo_market_data_enabled is rejected by configuration validation when app_env="production".

B. Binary Frame Decoding and Unit Normalization

The binary parser (decode_provider_frame) decodes big-endian packets for Transaction Code 1 (TC 1, last-traded-price header of 31 bytes) and Transaction Code 3

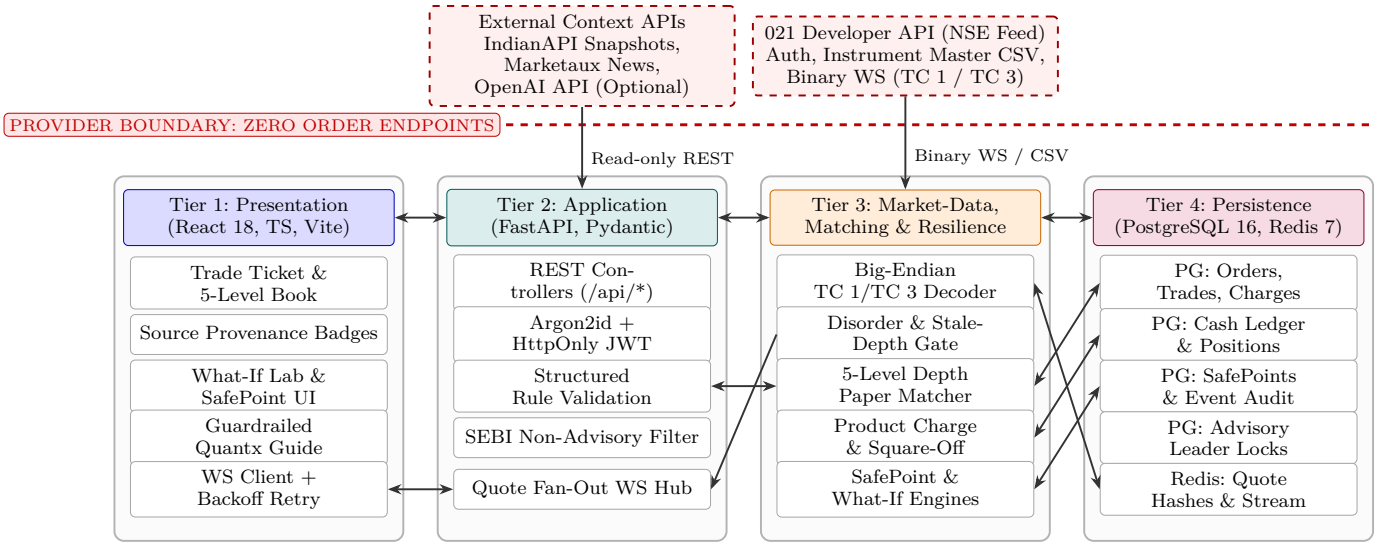


Fig. 1. Quantx four-tier system architecture illustrating the strict read-only provider boundary above the application, the presentation and FastAPI control tiers, the in-process market-data decoding, depth-matching, SafePoint recovery, and What-If resilience tier, and the authoritative PostgreSQL/Redis persistence tier.

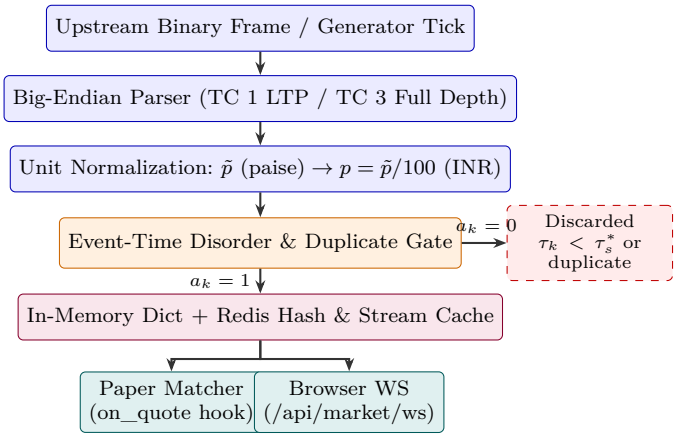


Fig. 2. Boundary-enforced market-data pipeline from upstream binary frame to the local paper matcher and browser WebSocket hub, highlighting paise-to-rupee normalization, event-time disorder and duplicate filtering, and dual memory/Redis caching.

(TC 3, full cash/derivative packet of 171 bytes or index packet of 47 bytes); heartbeat frames (TC 10) are discarded. In full subscription mode (subscribe_full), TC 3 packets for NSE cash equities carry ten 12-byte depth records—five bid levels followed by five ask levels—each containing unsigned 32-bit quantity, unsigned 32-bit price in paise, and unsigned 16-bit order count. Because provider packets and the NSE instrument master express all monetary values in integer paise whereas Quantx stores and displays rupee decimals, every price field is normalized at the ingestion boundary:

$$p = \frac{\tilde{p}}{100}, \tag{1}$$

where $\tilde{p} \in \mathbb{N}_0$ is the raw provider price or circuit bound (paise) and $p \in \mathbb{R}_{\geq 0}$ is the normalized rupee price (INR) quantized to $0.01\,INR$. Share quantities and order counts remain unitless non-negative integers.

C. Event-Time Disorder and Stale-Depth Guards

Each decoded TC 3 packet carries an unsigned 32-bit exchange timestamp in seconds relative to the NSE epoch offset ($\Delta_{\text{epoch}} = 315{,}513{,}000\,\text{s}$), which is converted to a UTC timestamp τ_k; packets lacking an exchange timestamp (such as TC 1 frames) are stamped with the server receipt time upon arrival. For any instrument symbol s, let τ_k denote the exchange timestamp of arriving packet k and τ_s^* denote the exchange timestamp of the currently cached quote. To prevent network reordering from regressing book state [13], [14], the ingestion filter evaluates the acceptance indicator a_k:

$$a_k = \begin{cases} 0, & \text{source} = \text{021_live},\ \tau_k, \tau_s^* \text{ defined},\ \tau_k < \tau_s^*, \\ 1, & \text{otherwise}, \end{cases} \tag{2}$$

where $a_k \in \{0, 1\}$ is the unitless acceptance decision and τ_k, τ_s^* are UTC exchange timestamps (s). Packets with equal timestamps ($\tau_k = \tau_s^*$) but updated book state are admitted so sub-second depth revisions are preserved. Independently of timestamp comparison, the service computes a structural quote signature comprising (data_source, price, volume, bids, asks) across all five bid and ask levels; any arriving 021_live or demo packet whose signature is identical to the cached signature for symbol s is suppressed as a duplicate before touching Redis or the paper matcher.

At order-submission and matching time, the accounting plane enforces a staleness gate:

$$A_s(t) = t - \tau_s \le \theta, \tag{3}$$

where $A_s(t)$ is the quote age at evaluation time t (s), τ_s is the stored quote's effective timestamp (exchange timestamp when present, otherwise receipt timestamp, in UTC s), t is the backend host's UTC wall-clock time (s), and θ is the configured staleness threshold market_quote_max_age_seconds (default $120\,s$). A missing or unparseable timestamp fails Equation (3) unconditionally. Any user-submitted MARKET order for symbol s when no quote is cached or when $A_s(t) > \theta$ is rejected immediately with HTTP 409, preventing market fills against frozen books. LIMIT, SL, and SL-M orders are admitted into the order book during stale intervals, but immediate matching is deferred until a fresh quote satisfying Equation (3) arrives.

D. Subscription Lifecycle and Multi-Worker Coordination

A single upstream WebSocket connection to the provider is shared across all connected browsers, enforcing the provider's cap of 100 concurrent instrument subscriptions. A background retention loop (_subscription_loop) automatically retains upstream subscriptions for any symbol referenced by an open paper order or non-zero position even when all browser tabs disconnect. To support multi-worker deployments safely without duplicate upstream connections or duplicate intraday square-offs, Quantx acquires PostgreSQL session-level advisory locks (pg_try_advisory_lock with keys 410021 for the market-data leader and 410022 for the operations worker). Accepted quotes are pushed to an in-process bounded queue (10,000 entries) consumed by a dedicated worker that invokes paper_engine.on_quote inside a worker thread and broadcasts JSON quote frames to subscribed browser WebSockets.

V. Local Paper Matching, Statutory Charges, SafePoint Recovery, and Scenario Engines

The accounting and resilience plane converts validated user intents into auditable ledger state transitions and provides non-destructive risk and recovery tools. Below we formalize each mechanism in execution order.

A. Order Admission and Conservative Cash Reservation

When create_order receives a request, it executes a deterministic eight-step admission gate:

1) Idempotency check: Queries orders.idempotency_key. If a matching row exists for the same account, the existing order is returned idempotently; if owned by a different account, HTTP 409 is raised [17].
2) Tradability check: Verifies that the target Instrument exists, has is_tradable=True, and holds a valid provider_token (unless demo_market_data_enabled is active).
3) Tick normalization: Converts the instrument's tick_size_paise to rupee tick $\Delta p = \max(\text{tick_size_paise}, 1)/100$ (INR) and rounds any supplied limit_price or trigger_price to the nearest valid tick via $\text{round}(p/\Delta p) \cdot \Delta p$.
4) Quantity, board-lot, and freeze validation: Verifies that quantity $q_o \in \mathbb{N}$ satisfies $1 \le q_o \le q_{\text{freeze}}$ and $q_o \bmod q_{\text{lot}} = 0$, where $q_{\text{lot}} = \max(\text{board_lot_quantity}, \text{lot_size}, 1)$ and q_{freeze} is the instrument's NSE freeze quantity.
5) Circuit-band validation: For orders carrying a limit or trigger price, verifies $p_{\text{lower}} \le p \le p_{\text{upper}}$, where p_{lower} and p_{upper} are the instrument's lower and upper circuit limits in INR.
6) Feed provenance and staleness gate: Rejects MARKET orders if the quote source is indianapi, if no quote exists, or if Equation (3) is violated (unless allow_stale_market=True is set by the internal 15:20 IST square-off worker).
7) Stop-trigger directional consistency: Against reference price p_{ref} (last traded price), verifies that a BUY stop has $p_{\text{trig}} > p_{\text{ref}}$ (and $p_{\text{lim}} \ge p_{\text{trig}}$ for SL), whereas a SELL stop has $p_{\text{trig}} < p_{\text{ref}}$ (and $p_{\text{lim}} \le p_{\text{trig}}$ for SL).
8) Buying-power and holdings check: For a BUY order, the engine computes a conservative cash reservation R_o:

$$\begin{aligned} R_o &= q_o\, p_o^{\text{est}}\,(1+\rho) + \beta_o, \\ \rho &= \Big(\max\{t_{\text{CNC}} + u_{\text{CNC}},\, t_{\text{MIS,S}} + u_{\text{MIS}}\} \\ &\qquad + \epsilon_{\text{X}} + \epsilon_{\text{S}} + \phi\Big)(1+g), \end{aligned} \tag{4}$$

where R_o is the reserved cash amount for order o (INR); $q_o \in \mathbb{N}$ is the order quantity (shares); p_o^{est} is the reservation reference price (INR), taken as the rounded limit_price if present, else the highest visible ask price $\max_i \pi_i^{\text{ask}}$, else the quote's last price; ρ is the conservative upper-bound statutory charge ratio (unitless); t_{CNC} and u_{CNC} are the delivery STT and stamp-duty ratios; $t_{\text{MIS,S}}$ and u_{MIS} are the intraday sell STT and buy stamp-duty ratios; ϵ_{X}, ϵ_{S}, ϕ, and g are the exchange, SEBI, account regulatory, and GST ratios defined in Equation (8); and β_o is the per-order brokerage estimate (INR). A BUY order is admitted only if available cash satisfies $M_{\text{cash}} - M_{\text{res}} \ge R_o$. For a delivery (CNC) SELL order, the engine verifies that settled holdings minus open CNC sell orders cover q_o. Any admission failure raises HTTP 409 or 422 without persisting an order row.

B. Five-Level Depth-Walking Matching

When an admitted order is evaluated against a fresh quote, let the opposite side of the five-level book consist of levels $i \in \{1, \dots, 5\}$ ordered best-price first (ascending ask prices $\pi_1 \le \cdots \le \pi_5$ for a BUY; descending bid prices

$\pi_1 \geq \cdots \geq \pi_5$ for a SELL), with displayed share quantities $Q_i \in \mathbb{N}_0$ after any account-level stress transformation (Equation (12)). For a LIMIT order (or a triggered SL order) with limit price p_{lim}, the marketable level set is $E = \{i : \pi_i \leq p_{\text{lim}}\}$ for a BUY and $E = \{i : \pi_i \geq p_{\text{lim}}\}$ for a SELL; for a MARKET order (or triggered SL-M), $E = \{1, \ldots, 5\}$. The matcher walks $i \in E$ sequentially and allocates fill quantity x_i:

$$x_i = \min\Big\{Q_i,\ \max\Big(0,\ q_{\text{rem}} - \sum_{j \in E,\, j<i} x_j\Big)\Big\}, \qquad i \in E, \tag{5}$$

where $x_i \in \mathbb{N}_0$ is the quantity filled at level i (shares) and $q_{\text{rem}} = q_o - q_{\text{filled}}$ is the order's remaining unfilled quantity entering the pass (shares). The total pass fill quantity f and volume-weighted average execution price $\bar{p}$ are:

$$f = \sum_{i \in E} x_i, \tag{6}$$

$$\bar{p} = \frac{1}{f} \sum_{i \in E} x_i\, \pi_i, \qquad f > 0, \tag{7}$$

where $f \in \mathbb{N}_0$ is measured in shares and $\bar{p}$ in INR/share (quantized half-up to $0.01\,INR$). Each level with $x_i > 0$ generates an individual Trade row with sequence index, execution price π_i, and level quantity x_i. If an order has validity IOC, any residual quantity $q_{\text{rem}} - f > 0$ after walking E is immediately cancelled and its remaining cash reservation is released; for DAY orders, the residual remains OPEN or PARTIALLY_FILLED for subsequent quote ticks.

C. Product-Aware Per-Fill Statutory Charges and Ledger Commit

For each level fill (x_i, π_i) with gross trade value $V = x_i\,\pi_i$ (INR), the fee routine (_calculate_fees) computes a six-component Indian statutory charge breakdown:

$$\begin{aligned} C = {} & b\,\beta + \big(\epsilon_{\text{X}} + \epsilon_{\text{S}} + \phi + t_{k,d} + u_{k,d}\big)V \\ & + g\Big(b\,\beta + (\epsilon_{\text{X}} + \epsilon_{\text{S}} + \phi)V\Big), \qquad V = x_i\,\pi_i, \end{aligned} \tag{8}$$

where C is the total statutory and brokerage charge for the fill (INR); V is the fill turnover (INR); $b \in \{0, 1\}$ equals 1 on the first fill of an order (filled_quantity = 0) and 0 on subsequent partial fills of the same order; β is the per-order brokerage (INR), equal to the account's custom commission_per_order if configured, otherwise $0.00\,INR$ for CNC and $\min\{20.00\,INR,\, 0.0003\,V\}$ for MIS; $\epsilon_{\text{X}} = 0.0000325$ (0.00325%) is the NSE exchange transaction charge ratio; $\epsilon_{\text{S}} = 0.000001$ (0.0001%) is the SEBI turnover fee ratio; $\phi = \text{regulatory_fee_percent}/100$ is the account's additional regulatory fee ratio; $t_{k,d}$ is the STT ratio for product $k \in \{\text{CNC}, \text{MIS}\}$ and side $d \in \{\text{BUY}, \text{SELL}\}$ (0.001 on both sides for CNC; 0.00025 on SELL only for MIS); $u_{k,d}$ is the stamp-duty ratio (0.00015 on CNC BUY; 0.00003 on MIS BUY; 0 on SELL); and $g = 0.18$ (18%) is the GST rate applied to brokerage, exchange fees, and SEBI/regulatory fees. Each of the six components (brokerage, stt, exchange_fee, sebi_fee, stamp_duty, gst) is individually quantized half-up to $0.01\,INR$ before summing into C.

The account's cash balance M_{cash} changes by:

$$\Delta M = -\,d_{\pm}\,x_i\,\pi_i - C, \tag{9}$$

where ΔM is the net cash change (INR) and $d_{\pm} = +1$ for a BUY and -1 for a SELL. The engine writes two immutable CashLedger rows per fill—a TRADE_DEBIT or TRADE_CREDIT row for $\mp x_i \pi_i$ (keyed by trade-gross:<trade_id>) and a CHARGE row for $-C$ (keyed by trade-charge:<trade_id>)—and commits the Order, Trade, TradeCharge, OrderEvent, Position, and CashLedger mutations within a single PostgreSQL transaction [4].

D. Signed Positions and Charge-Capitalized Realized P&L

Statutory charges are capitalized into an effective per-share execution price $p_{\text{eff}} = \pi_i + d_{\pm}\,C/x_i$ (INR/share). For an existing position with signed share quantity $Q \in \mathbb{Z}$ ($Q > 0$ long, $Q < 0$ short) and average cost basis $A \in \mathbb{R}_{\geq 0}$ (INR/share), a fill that opens or adds to the same direction updates the cost basis and signed quantity as:

$$A' = \frac{|Q|\,A + x_i\,p_{\text{eff}}}{|Q'|}, \qquad Q' = Q + d_{\pm}\,x_i, \tag{10}$$

where A' is the new average cost basis (INR/share) and Q' is the updated signed position quantity (shares). A fill opposing the current position direction closes $c = \min(|Q|, x_i)$ shares and increments cumulative realized profit and loss by:

$$P_{\text{real}} = \text{sgn}(Q)\,(p_{\text{eff}} - A)\,c, \tag{11}$$

where P_{real} is the realized profit and loss (INR) and $\text{sgn}(Q) \in \{+1, -1\}$ is the sign of the prior position. If the fill flattens the position ($Q' = 0$), A' resets to 0; if the fill flips the sign of Q ($x_i > |Q|$), the residual $x_i - |Q|$ opens a new position in direction $d_{\pm}$ at average basis $A' = p_{\text{eff}}$.

E. Market Stress Mode, Multi-Modal What-If Lab, and Transactional SafePoint Recovery

1) Account-Level Market Stress Mode: When a learner enables Stress Mode on their account, simulated execution depth is degraded exclusively for that account prior to Equation (5):

$$\begin{aligned} \pi_i' &= \pi_i + s\,p_{\text{last}} \ \text{(asks)}, \\ \pi_i' &= \max\{0.01,\ \pi_i - s\,p_{\text{last}}\} \ \text{(bids)}, \\ Q_i' &= \lfloor \lambda\,Q_i \rfloor, \end{aligned} \tag{12}$$

where π_i' (INR) and Q_i' (shares) are the stressed level price and quantity, p_{last} is the quote's last price (INR), s is paper_stress_spread_percent (default 0.005, shifting each side by 0.5% of p_{last}), and λ is paper_stress_liquidity_factor (default 0.5, halving displayed depth).

2) Multi-Modal What-If Scenario Laboratory: Quantx provides two complementary, strictly read-only scenario engines that project hypothetical shocks without mutating ledger state. First, the portfolio holding shock engine (/api/scenarios) evaluates symbol-specific percentage shocks δ_j (%) across each held position j with quantity q_j (shares) and current price p_j (INR):

$$V_j^{\text{proj}} = q_j \max\Big\{0,\ p_j\Big(1+\frac{\delta_j}{100}\Big)\Big\}, \qquad \Pi_j = V_j^{\text{proj}} - q_j\, p_j, \tag{13}$$

where V_j^{proj} is the projected holding valuation (INR) and Π_j is the projected holding P&L delta (INR). Second, the multi-modal What-If laboratory (ScenarioEngine, /api/what-if/*) supports three interactive analytical modes: (i) Single-Stock Scenario Grid, which evaluates a 3×3 matrix of price moves $\Delta p \in \{-5\%, 0\%, +5\%\}$ and volatility shifts $\Delta\sigma \in \{-10\%, 0\%, +10\%\}$ around user-specified moves; (ii) Portfolio Stress Test, which combines a global market shift δ_{mkt} (%), a sector shift δ_{sec} (%), and stock overrides; and (iii) Capital Allocation Concentration Analysis, which computes portfolio Herfindahl–Hirschman concentration and top-holding weight $w_{\max} = \max_j(V_j/\sum_k V_k)$, flagging high concentration risk when $w_{\max} > 0.35$.

3) Transactional SafePoint Recovery Shield: To allow learners to experiment with aggressive order structures or stress scenarios and recover cleanly, RecoveryEngine (/api/recovery/*) implements atomic portfolio checkpointing and autonomous fault recovery. Creating a SafePoint S_m captures a serialized JSON state vector:

$$S_m = \big(M_{\text{cash}}^{(m)},\ M_{\text{res}}^{(m)},\ \mathcal{P}^{(m)},\ \mathcal{O}_{\text{open}}^{(m)}\big), \tag{14}$$

where $M_{\text{cash}}^{(m)}$ and $M_{\text{res}}^{(m)}$ are the account's cash and reserved balances (INR) at capture time t_m, $\mathcal{P}^{(m)} = \{(i_r, k_r, Q_r, A_r, P_{\text{real},r})\}_{r=1}^{R_m}$ is the set of active position tuples, and $\mathcal{O}_{\text{open}}^{(m)}$ is the set of open orders. Before executing a rollback, the user can request a dry-run diff (/api/recovery/diff), which computes the exact cash restoration delta and per-symbol quantity adjustment:

$$\Delta M_{\text{restore}} = M_{\text{cash}}^{(m)} - M_{\text{cash}}(t), \quad \Delta Q_s = Q_s^{(m)} - Q_s(t), \tag{15}$$

where $\Delta M_{\text{restore}}$ (INR) is the cash balance adjustment and ΔQ_s (shares) is the position delta for symbol s. Executing rollback (execute_rollback) runs inside a single database transaction: it cancels all currently OPEN, PENDING, or PARTIALLY_FILLED orders, restores $M_{\text{cash}} \leftarrow M_{\text{cash}}^{(m)}$ and $M_{\text{res}} \leftarrow 0.00\,INR$, replaces current positions with $\mathcal{P}^{(m)}$, and appends an auditable RESET entry to cash_ledger. Furthermore, an autonomous recovery daemon (simulate_fault_and_recover) detects three classes of simulated runtime faults—DATA_CORRUPTION (non-numeric price injection), OVERLOAD (500-order burst congestion), and CONCURRENCY (conflicting state mutation)—and automatically executes a transactional rollback to the latest healthy SafePoint. Fig. 3 illustrates the SafePoint Recovery Shield and Multi-Modal What-If pipeline.

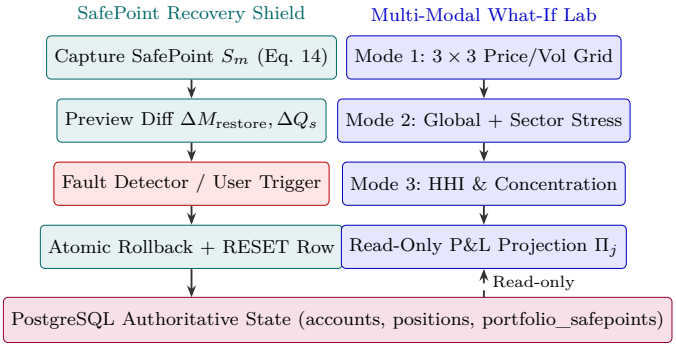


Fig. 3. Dual resilience architecture showing (left) the transactional SafePoint Recovery Shield that captures portfolio snapshots, computes pre-rollback diffs, and executes atomic state rollbacks, and (right) the strictly read-only Multi-Modal What-If Scenario Laboratory.

VI. Runtime Configuration, Structured Validation, and Guardrailed Assistance

Every paper order traverses a deterministic four-phase execution pipeline, illustrated in Fig. 4. In Phase 1 (Ingestion & Normalization), the backend authenticates the HTTP/WebSocket session, decodes incoming TC 1/TC 3 binary frames, normalizes paise to INR, and parses the order payload. In Phase 2 (Rule Gate & Cash Reservation), the admission gate enforces idempotency, tradability, tick rounding, board-lot and freeze bounds, circuit bands, quote provenance and staleness ($\theta = 120\,s$), stop-trigger direction, and conservative cash reservation R_o; any violation aborts execution and returns a structured HTTP 409/422 error with zero state mutation. In Phase 3 (Depth-Walking Matching & Statutory Charges), the matcher walks up to five book levels, handles IOC remainders, and computes the six-component product-aware statutory charge vector. In Phase 4 (Transactional Commit & Real-Time Sync), the engine writes all order, trade, charge, event, position, and cash-ledger records inside a single PostgreSQL transaction and pushes updated state to the client workspace.

TABLE I documents the runtime configuration parameters across all backend modules, and TABLE II specifies the validated order-submission contract. The complete API and database schemas are provided in Appendix A.

A. Order Lifecycle, Idempotency, and Guardrailed Quantx Guide

Fig. 5 depicts the order-status state machine combined with the SafePoint rollback transition. Admitted orders enter OPEN (or reject at the HTTP gate without creating a database row), transition through PARTIALLY_FILLED to FILLED as depth levels are con-

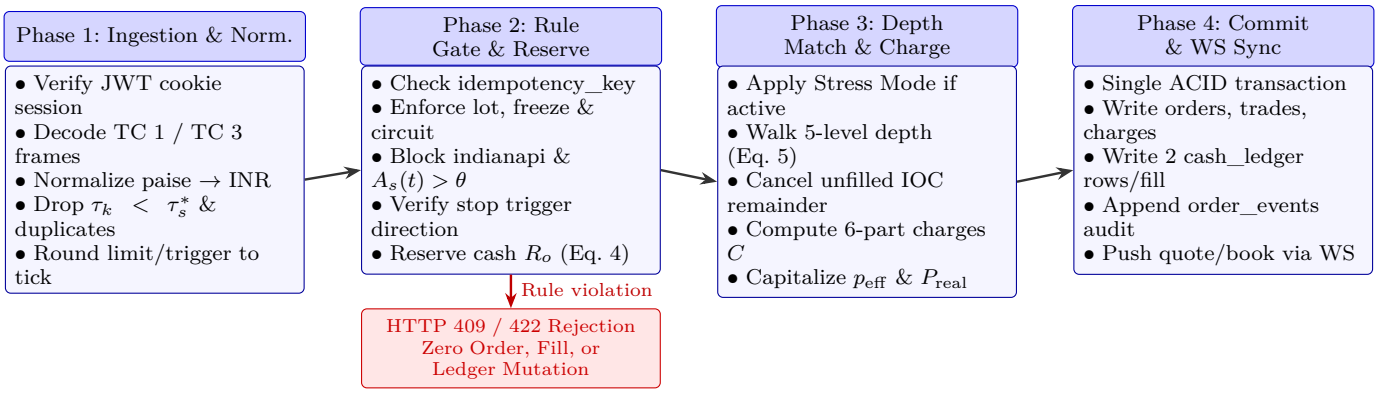


Fig. 4. Four-phase Quantx order-execution workflow detailing session and frame ingestion, the eight-point admission and cash-reservation rule gate, five-level depth-walking matching with Indian statutory charge computation, and single-transaction PostgreSQL persistence with WebSocket synchronization.

TABLE I
Module Configuration Parameters

Module	Configured values in active codebase
Market feed	Full mode (subscribe_full); 5 bid / 5 ask levels; TC 1 (31 B) and TC 3 (171 B/47 B) decoded; TC 10 ignored; max 100 concurrent instruments; advisory lock 410021
Staleness guard	θ = market_quote_max_age_seconds, default 120 s
Statutory charges†	Exchange $\epsilon_X = 0.0000325$; SEBI $\epsilon_S = 0.000001$; GST $g = 0.18$ on $(\beta + \epsilon_X + \epsilon_S + \phi)$; CNC: STT 0.001 both sides, stamp 0.00015 buy, brokerage 0 INR; MIS: STT 0.00025 sell, stamp 0.00003 buy, brokerage $\min\{20\,INR, 0.0003V\}$ on first fill; custom account commission/regulatory fee supported
Stress mode	Spread multiplier $s = 0.005$ of p_{last}; liquidity factor $\lambda = 0.5$; per-account toggle
MIS square-off	Weekdays $\geq$ 15:20 IST, polled every 10 s; idempotent key auto-squareoff:<date>:<pos_id>; advisory lock 410022
Capital	Allowed presets from INR 100,000 to 5,000,000; SafePoint JSON snapshots in portfolio_safepoints
What-If Lab	Single-stock 3×3 price/volatility grid; portfolio global/sector stress; concentration risk threshold $w_{\max} > 0.35$
Quantx Guide	Dual-mode (market_guide / support); regex + prompt guardrails blocking buy/sell investment advice; max 6 history turns
Auth & recovery	Argon2id password hash; HttpOnly, SameSite=Lax JWT cookie; SHA-256 reset token (30 min TTL, 1 req/min throttle)

†Statutory ratios in PaperTradingEngine._calculate_fees; PAPER_* environment settings parameterize the conservative reservation ratio ρ in Equation (4).

TABLE II
Validated Order Request Fields (OrderCreateInput)

Field	Purpose and enforced validation constraint
instrument_id	Instrument primary key (UUID string); must reference an active, tradable NSE instrument
side	Order direction: BUY or SELL
product	CNC (cash delivery) or MIS (intraday square-off); governs STT, stamp duty, brokerage, and 15:20 IST auto-exit
order_type	MARKET, LIMIT, SL (stop-limit), or SL-M (stop-market)
quantity	Integer in [1, 1,000,000] shares; exact multiple of board lot q_{lot}; bounded by $q_o \leq q_{\text{freeze}}$
limit_price	Positive decimal (INR); required for LIMIT/SL, forbidden for MARKET/SL-M; tick-rounded; inside $[p_{\text{lower}}, p_{\text{upper}}]$
trigger_price	Positive decimal (INR); required for SL/SL-M, forbidden otherwise; tick-rounded; directional stop check vs. p_{last}
validity	DAY (default) or IOC (unfilled remainder cancelled immediately)
idempotency_key	String of 8–64 characters; unique across orders; same-account repeat returns existing order, cross-account raises HTTP 409

sumed, or transition to CANCELLED via user cancellation, IOC remainder expiry, or a SafePoint rollback. Every state transition appends a structured JSON record to order_events (ACCEPTED, MODIFIED, TRIGGERED, PARTIAL_FILL, FILL, FILLED, CANCELLED).

Finally, the workspace includes the Quantx Guide (/api/support/chat), a dual-mode educational and support assistant operating in market_guide mode (explaining NSE market sessions, pre-open 09:00–09:15 IST, regular session 09:15–15:30 IST, circuit breakers, and

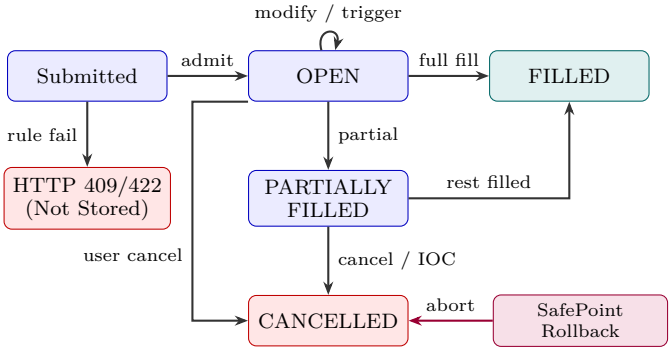


Fig. 5. Paper-order lifecycle and SafePoint rollback state machine showing admission, stop triggering, partial fills, IOC remainder cancellation, and atomic cancellation of open orders during SafePoint restoration.

product mechanics) and support mode (troubleshooting HTTP 409/422 order rejections, staleness blocks, and SafePoint recovery). To enforce SEBI regulatory boundaries between educational software and registered investment advisers, the service passes every user query through a deterministic advisory-intent filter (_asks_for_investment_advice) prior to knowledge retrieval or optional LLM formatting, refusing any request for buy/sell recommendations, target prices, or stock tips while offering to explain the underlying order-book or charge mechanics instead.

VII. Experimental Setup and System Evaluation

We evaluated Quantx through a controlled, reproducible replay and microbenchmark suite executed directly against the containerized PostgreSQL 16.15 and Redis 7.4.11 stack. The primary goal is to audit whether the implemented pipeline satisfies Hypotheses H1–H3 and to quantify where a naive simulator baseline (B0) fails. TABLE III documents the exact hardware and software environment used for all measurements.

Provenance and Audit of Recorded Input. The repository includes backend/data/recorded_market_ticks.jsonl (80,504 records across 100 NSE symbols, 804–806 records per symbol, spanning 652.7 s with median inter-arrival 6.3 ms and p99 26.3 ms) and quote_snapshots.json (100 symbols). Forensic inspection of the codebase and git history (f89c99b) resolves the exact provenance of these files: record_market_tick is invoked exclusively by MarketDataStore.update_quote when source=="live", without passing an exchange_time argument (so exchange_time is null on all 80,504 lines). Furthermore, receipt timestamps span 15:13:02Z–15:23:55Z (20:43:02–20:53:55 IST), which lies outside NSE's regular trading session (09:15–15:30 IST), and the records contain no five-level bid/ask arrays, exhibit cyclic 100-symbol emission, include 151 zero-price ticks, and violate tick-size multiples on 30.9% of positive prices. Consequently, we use recorded_market_ticks.jsonl strictly as an empirical inter-

TABLE III
Evaluation Hardware and Software Environment

Component	Verified specification
Host CPU	Intel Core i5-8500T @ 2.10 GHz (6 physical cores, 6 hardware threads)
Memory & storage	8.0 GB DDR4 RAM (7.78 GB usable); 512 GB WD PC SN5000S NVMe SSD
Operating system	Microsoft Windows 11 Pro (64-bit) with WSL2 Linux kernel backend
Container runtime	Docker Engine v29.9.2, Docker Compose v5.6.0
Backend runtime	Python 3.12.15, FastAPI 0.156.8, Pydantic 2.14.1, SQLAlchemy 2.1.5, psycopg 3.3.6, uvicorn 0.54.0, argon2-cffi 25.1.0
Frontend runtime	Node.js v24.19.0, React 18, TypeScript, Vite
Persistence stack	PostgreSQL 16.15 (postgres:16-alpine); Redis 7.4.11 (redis:7-alpine)
Recorded tick corpus	recorded_market_ticks.jsonl (80,504 lines, SHA-256 prefix a9fd8616); committed in initial snapshot f89c99b

arrival and anomaly corpus, and construct deterministic five-level depth frames (source="021_live") across liquid NSE instruments (RELIANCE, TCS, HDFCBANK, INFY, SBIN) for controlled depth-walking and disorder evaluation.

Controlled Scenario Profiles and Baseline B0. Both Quantx and baseline B0 were initialized with identical virtual accounts (5,000,000.00 INR starting capital) and subjected to $N = 50$ valid scripted orders across three scenario profiles plus $N_{\text{adv}} = 50$ adversarial fault events:

1) Typical Profile ($N_1 = 20$ orders): Marketable BUY and SELL orders (10 CNC, 10 MIS; MARKET and LIMIT) with quantities $q_o \in [8, 30]$ shares that fill completely within the best displayed level ($Q_1 = 100$ shares).
2) Boundary Profile ($N_2 = 20$ orders): Multi-level depth-walking orders ($q_o \in [60, 250]$ shares across five levels of $Q_i = 50$ shares each, generating $N_f = 73$ total level fills across the $N = 50$ valid orders), IOC orders with partial fills and immediate remainder cancellation, prices placed exactly on circuit and tick boundaries, and triggered SL/SL-M orders.
3) Adversarial Profile ($N_3 = 10$ valid orders + $N_{\text{adv}} = 50$ injected faults): Valid multi-level limit orders interleaved with 50 adversarial events: (i) 16 out-of-order provider packets with $\tau_k < \tau_s^*$ and 8 duplicate packets, (ii) 8 MARKET orders submitted against stale quotes ($A_s(t) = 300\,s > \theta$), (iii) 7 repeated submissions of an existing idempotency_key, and (iv) 11 rule-violating payloads (exceeding NSE freeze quantity, violating board-lot multiples, breaching upper/lower circuit bands, violating stop-trigger direction, or exceeding available buying power).

Baseline B0 implements the legacy naive simulator rules: it executes the full order quantity q_o at the last traded price p_{last} in a single fill, charges $0.00\,INR$ in statutory fees, overwrites cached quotes in arrival order regardless of τ_k, applies no staleness check, ignores idempotency_key, and omits circuit, lot, freeze, and buying-power gates. Expected reference fills, statutory charges, and cash balances (M_n^{ref}) are computed by an independent reference oracle (OracleEngine) implementing Equations (1)–(9) with identical half-up paisa quantization.

A. Correctness and Resilience Audit

TABLE IV presents the empirical correctness audit comparing B0 and Quantx against the independent reference oracle across all $N = 50$ valid orders ($N_f = 73$ level fills in Quantx) and $N_{\text{adv}} = 50$ adversarial injections. At the end of the 50-order sequence, Quantx's authoritative PostgreSQL cash balance was $2{,}242{,}498.88\,INR$, matching the independent oracle balance of $2{,}242{,}498.88\,INR$ with an exact reconciliation difference of $0.00\,INR$ ($\varepsilon_M = 0.00\,INR \leq 0.01\,INR$, confirming H1). By contrast, B0 accumulated a mean cumulative ledger error of $19{,}684.41\,INR/order$ (maximum $227{,}398.32\,INR$) due to executing partially marketable limit orders in full at p_{last}, ignoring five-level depth slippage ($\delta_p = 1.60\,INR/share$ mean, $9.51\,INR/share$ max), and omitting statutory charges ($33.08\,INR/fill$ mean, $116.01\,INR/fill$ max). Under the Adversarial profile, Quantx recorded zero stale market-order executions, zero older-timestamp quote overwrites ($O = 0$, confirming H2), zero duplicate effects from repeated idempotency keys, and zero accepted invalid orders (confirming H3).

B. Latency and Throughput Telemetry

To characterize runtime overhead, each core operation was executed $N_{\text{op}} = 200$ times after a 20-iteration warm-up (50 measurement iterations after 10 warm-ups for full SafePoint capture-and-rollback cycles) using Python's monotonic high-resolution performance counter (time.perf_counter_ns). TABLE V reports the arithmetic mean, median (p50), 95th percentile (p95), and 99th percentile (p99) latencies in milliseconds.

In-memory quote ingestion and disorder filtering operate in $0.04\,ms$ (p99 $0.06\,ms$), and WebSocket serialization and fan-out complete in $0.15\,ms$ median (p99 $0.33\,ms$). Write-intensive operations are bounded by synchronous PostgreSQL transactional durability on the containerized host: order admission requires $13.77\,ms$ at p50, a single-level market fill and ledger commit requires $26.83\,ms$ at p50 (p99 $70.26\,ms$), and a full five-level depth walk—which inserts five Trade rows, five TradeCharge rows, ten CashLedger rows, and six OrderEvent rows in one transaction—completes in $71.02\,ms$ at p50 and $118.02\,ms$ at p99. Capturing a SafePoint and executing a complete transactional rollback requires $22.00\,ms$ at p50 (p99 $29.45\,ms$).

TABLE IV
Correctness and Resilience Audit on Scripted Scenarios ($N = 50$ Valid Orders, $N_f = 73$ Fills, $N_{\text{adv}} = 50$ Faults)

Metric (unit)	B0	Quantx	Diff.
Ledger error, mean (INR/order)	19,684.41	0.00	−19,684.41
Ledger error, max (INR/order)	227,398.32	0.00	−227,398.32
Fill-price dev., mean (INR/share)	1.60	0.00	−1.60
Fill-price dev., max (INR/share)	9.51	0.00	−9.51
Charge dev., mean (INR/fill)	33.08	0.00	−33.08
Charge dev., max (INR/fill)	116.01	0.00	−116.01
Fills beyond limit price (count)	9	0	−9
Executions on stale depth (count)‡	8	0	−8
Older-packet overwrites (count)‡	16	0	−16
Extra effects of repeated key (count)‡	7	0	−7
Invalid orders accepted (count)‡	10	0	−10

Measured on live PostgreSQL 16.15 and Redis 7.4.11 via run_paper_benchmarks.py. ‡Evaluated across the $N_{\text{adv}} = 50$ adversarial fault injections.

C. Contextual Comparison

TABLE VI separates properties empirically verified in TABLE IV and TABLE V from architectural design invariants and unassessed external dimensions.

VIII. Discussion, Limitations, and Ethical Boundaries

Controlled Synthetic Benchmark Scope. The correctness evaluation in Section VII compares Quantx and B0 against a reference oracle that encodes the same mathematical specification (Equations (1)–(9)). Consequently, while the audit proves that the production PostgreSQL implementation conforms to the specification to within $0.00\,INR$ and exposes the large accounting errors of a naive simulator, it does not measure specification fidelity against live NSE matching engines. Threats to validity follow standard software-engineering taxonomy [22]: internal validity is high due to deterministic inputs and isolated database transactions, whereas external validity is bounded by the scripted scenario profiles.

Recorded-Input Provenance and Limitations. As established in Section VII, the repository's recorded_market_ticks.jsonl file was generated outside NSE market hours via market_store.update_quote(..., source="live") without exchange timestamps or five-level depth arrays, and exhibits synthetic cyclic emission and off-tick prices. Evaluating multi-level depth walking against real historical exchange books therefore requires

TABLE V
Per-Operation Latency Distribution Across Core Subsystem Paths ($N_{\text{op}} = 200$ Repetitions After 20 Warm-Ups, in ms)

Operation	Mean (ms)	p50 (ms)	p95 (ms)	p99 (ms)	Execution path and persistence scope
Quote ingest (decode + disorder guard)	0.04	0.04	0.04	0.06	TC 3 frame normalization, Eq. (2) check, in-memory cache update
Order admission (rule gate + cash reserve)	14.43	13.77	19.07	22.68	8-point validation, Eq. (4) reserve, PostgreSQL orders/events commit
Match and commit — single level (1 fill)	28.48	26.83	36.88	70.26	Admission + 1-level depth match + 6-part charge + 2 ledger rows + PG commit
Match and commit — full 5-level walk (5 fills)	72.76	71.02	87.04	118.02	Admission + 5 separate Trade/TradeCharge rows + 10 ledger rows + PG commit
Browser WebSocket fan-out (cache to client)	0.17	0.15	0.22	0.33	FastAPI jsonable_encoder serialization + loopback WebSocket dispatch
Portfolio summary read (/api/portfolio/summary)	4.36	3.91	6.13	8.48	Account + active positions + unrealized P&L aggregation from PostgreSQL
SafePoint capture + atomic rollback cycle	22.65	22.00	28.93	29.45	JSON snapshot creation + open-order cancellation + position/ledger restoration

an authenticated TC 3 binary capture recorded during live NSE market hours (09:15–15:30 IST), as formalized in Protocol D.

Resolved Code–Documentation Discrepancies. Auditing the full repository surfaced five architectural discrepancies between legacy files, project documentation, and the active application entry point (backend/app/main.py):

1) Dormant legacy modules: The repository contains routes/orders.py, services/matching_engine.py, services/charge_engine.py, and services/square-off_service.py. None of these four modules is imported by main.py; indeed, they attempt to import OrderFill from app.models, a class absent from the current SQLAlchemy schema. All active order, matching, charge, and square-off traffic runs through routes/trading.py and services/paper_engine.py.
2) Hardcoded vs. environment charge constants: While config.py defines PAPER_* charge ratios, those settings feed only the conservative reservation formula (Equation (4)); actual posted trade charges in PaperTradingEngine._calculate_fees use hardcoded constants (and the dormant charge_engine.py uses a third set of constants: 0.05% CNC brokerage, 0.0000345 exchange fee, and zero SEBI fee).
3) Dual quote stores for What-If vs. Trading: Live quotes decoded by MarketDataService populate market_data.quotes and Redis, which feed PaperTradingEngine and /api/scenarios. However, ScenarioEngine (/api/what-if/*) reads prices from the in-memory market_store singleton (seeded from quote_snapshots.json and 15 default symbols), which is not updated by MarketDataService._record_quote.
4) Dual stress-mode implementations: Account-level Stress Mode in PaperTradingEngine widens bid/ask levels by 0.5% of p_{last} and halves displayed quantities (Equation (12)), whereas market_store.get_quote contains a legacy sinusoidal price perturbation read only by ScenarioEngine.
5) Unused schema enumerations: Although Order.status permits PENDING_TRIGGER, PENDING, REJECTED, and EXPIRED, the active engine admits stop orders directly as OPEN, rejects invalid orders via HTTP exceptions before row insertion, and contains no end-of-day expiration worker for unfilled DAY orders.

Single-Process Scaling and Fill-Fidelity Boundaries. Although PostgreSQL advisory locks safely coordinate a single upstream feed reader across web workers, PaperTradingEngine.on_quote acquires a process-wide thread lock and iterates synchronously over all accounts with open orders for the updated symbol, meaning matching latency scales linearly with active account count. Furthermore, matching consumes at most five displayed levels, book depletion is not shared across concurrent learner accounts on the same tick, intraday MIS SELL orders do not enforce a short-margin reservation check, and 4,433 of the 9,939 NSE cash instruments in the bundled master carry a board lot of 100 shares. No claim is made regarding queue priority,

TABLE VI
Measured and Contextual System Comparison

Property	B0	Quantx	Evidence classification
Fill price basis	Last price	5-level depth walk	Measured (Table IV: $\delta_p = 0.00\,INR$)
Statutory charges	None	6-part product-aware	Measured (Table IV: $0.00\,INR$ dev.)
Packet ordering	Arrival order	Event-time $\tau_k \geq \tau_s^*$	Measured (Table IV: 0 overwrites)
Stale-depth gate	Unchecked	Blocked ($A_s(t) > \theta$)	Measured (Table IV: 0 stale fills)
State recovery	None	Atomic SafePoints	Measured (Table V: p50 = $22.00\,ms$)
Orders to provider	No	No	Contextual (enforced by architecture)
Advisory guardrail	None	Regex + prompt block	Contextual (verified by unit/API code)
Live-exchange fidelity	Not assessed	Not assessed	Contextual (deferred to Protocol D)
Learning outcomes	Not assessed	Not assessed	Contextual (out of scope; no user trial)

hidden iceberg liquidity, or price impact.

Regulatory and Ethical Boundary. Quantx is strictly an educational simulation workspace and not a securities broker, exchange member, or SEBI-registered investment adviser. All funds, fills, and SafePoint recoveries are virtual, and the Quantx Guide enforces deterministic non-advisory guardrails. Because simulated profits are susceptible to selection bias and lack real financial consequence [20], simulated performance in Quantx must never be interpreted as predictive of live trading returns.

IX. Conclusion and Reproducible Future Evaluation Protocols

We designed, implemented, and empirically audited Quantx, a four-tier NSE equity paper-trading workspace that enforces a strict read-only provider boundary while executing five-level depth-walking matching, six-component Indian statutory charge accounting, event-time disorder filtering, non-destructive What-If stress testing, and transactional SafePoint state recovery inside an authoritative PostgreSQL ledger. Across $N = 50$ scripted orders ($N_f = 73$ fills) and $N_{\text{adv}} = 50$ injected adversarial faults, Quantx confirmed Hypotheses H1–H3 with $0.00\,INR$ ledger-reconciliation error, $0.00\,INR/share$ fill-price deviation, zero stale or out-of-order state corruptions, sub-millisecond quote ingestion (p99 = $0.06\,ms$), and $22.00\,ms$ median SafePoint rollback latency. To support independent extension of this work [21], we define four pre-registered future evaluation protocols:

Protocol A — Multi-Account Ledger and Fill Conformance. Units: $N \geq 500$ scripted orders across Typical, Boundary, and Adversarial profiles replayed deterministically. Metrics: Maximum absolute cash reconciliation error ε_M and mean fill-price deviation δ_p:

$$\varepsilon_M = \max_{1 \leq n \leq N} \left| M_n^{\text{rec}} - M_n^{\text{ref}} \right|, \quad \delta_p = \frac{1}{N_f} \sum_{n=1}^{N_f} \left| \bar{p}_n^{\text{rec}} - \bar{p}_n^{\text{ref}} \right|, \tag{16}$$

where $M_n^{\text{rec}}, M_n^{\text{ref}}$ are recorded and oracle cash balances (INR) and $\bar{p}_n^{\text{rec}}, \bar{p}_n^{\text{ref}}$ are recorded and oracle fill prices (INR/share) across N_f fills. Success criterion: $\varepsilon_M \leq 0.01\,INR$, $\delta_p = 0.00\,INR/share$, and zero fills violating limit bounds.

Protocol B — High-Rate Feed-Disorder and Recovery Invariance. Units: Replay streams of 10^5 TC 3 packets with 10% injected timestamp inversions ($\tau_k < \tau_s^*$), 10% exact duplicates, and periodic simulated DATA_CORRUPTION faults. Metric: Stale cache overwrite count O:

$$O = \sum_{k=1}^{10^5} \mathbb{1}[\tau_k < \tau_s^* \ \wedge \ \text{cache updated by packet } k], \tag{17}$$

where O is unitless. Success criterion: $O = 0$ across all replays and 100% exact state restoration ($\Delta M_{\text{restore}} = 0.00\,INR$ post-rollback) after every injected fault.

Protocol C — Concurrent Load Latency and Throughput Scaling. Units: Operations across offered loads of $\Lambda \in \{10, 50, 100, 250\}$ concurrent virtual accounts submitting orders over HTTP/WebSocket. Metric: Order-statistic latency quantiles ℓ_q:

$$\ell_q = x_{(\lceil q N_{\text{op}} \rceil)}, \qquad q \in \{0.50,\ 0.95,\ 0.99\}, \tag{18}$$

where $x_{(j)}$ is the j-th order statistic of N_{op} measured latencies (ms). Success criterion: Five-level match-and-commit $\ell_{0.99} \leq 250.0\,ms$ at $\Lambda = 100$ concurrent accounts with zero dropped WebSocket frames or transaction deadlocks.

Protocol D — Live-Session Exchange Depth Fidelity Assessment. Units: Historical paper orders replayed against licensed, exchange-timestamped five-level NSE cash-equity depth captured during regular market hours (09:15–15:30 IST). Metric: Signed execution price deviation in basis points Δ_{bp}:

$$\Delta_{\text{bp}} = 10^4 \, \frac{\bar{p}^{\text{sim}} - \bar{p}^{\text{obs}}}{\bar{p}^{\text{obs}}}, \tag{19}$$

where $\bar{p}^{\text{sim}}$ and $\bar{p}^{\text{obs}}$ are simulated and observed volume-weighted average execution prices (INR/share). Success criterion: Median $|\Delta_{\text{bp}}| \leq 5.0\,bp$ for orders within displayed five-level depth.

Appendix A
Order, Scenario, Recovery, and Database Schemas

Verified REST/WebSocket request payloads and core PostgreSQL table constraints from backend/app/schemas.py and backend/app/models.py:

```
POST /api/orders
{
  "instrument_id": "6f9b2c10-...",
  "side": "BUY", // BUY | SELL
  "product": "CNC", // CNC | MIS
  "order_type": "LIMIT", // MARKET | LIMIT | SL | SL-M
  "quantity": 50, // 1..1000000, multiple of lot
  "limit_price": "1395.50", // INR decimal, rounded to tick
  "trigger_price": null, // Required for SL and SL-M
  "validity": "DAY", // DAY | IOC
  "idempotency_key": "ord-2026-0001"
}

POST /api/what-if/portfolio-stress
{
  "market_Shock": -5.0, // Global shift (%)
  "sector_shocks": {"IT": -8.0}, // Sector shift (%)
  "stock_shocks": {"TCS": -10.0} // Symbol override (%)
}

POST /api/recovery/safepoints
{ "name": "Pre-Volatile-Breakout" }

POST /api/recovery/rollback
{ "safepoint_id": 1 }

WS /api/market/ws (Client -> Server)
{ "task": "subscribe", "symbols": ["RELIANCE", "TCS"] }
```

Authoritative PostgreSQL relational schema constraints (backend/app/models.py):

```
accounts: id UUID PK, user_id UUID UNIQUE FK,
  initial_capital NUMERIC(14,2) CHECK (>0),
  cash_balance NUMERIC(14,2) CHECK (>=0),
  reserved_cash NUMERIC(14,2) CHECK (>=0),
  stress_mode BOOLEAN DEFAULT FALSE,
  commission_per_order NUMERIC(10,2),
  regulatory_fee_percent NUMERIC(8,6)
instruments: id UUID PK, UNIQUE(symbol, exchange),
  lot_size INT (>0), board_lot_quantity INT (>0),
  freeze_quantity INT, tick_size_paise INT (>0),
  lower_circuit_paise INT, upper_circuit_paise INT,
  provider_token INT, is_tradable BOOLEAN
orders: id UUID PK, account_id UUID FK, instrument_id UUID FK,
  side VARCHAR(8) IN ('BUY','SELL'),
  product VARCHAR(16) IN ('CNC','MIS'),
  order_type VARCHAR(16) IN ('MARKET','LIMIT','SL','SL-M'),
  quantity INT (>0), filled_quantity INT (0..quantity),
  limit_price NUMERIC(14,2), trigger_price NUMERIC(14,2),
  avg_exec_price NUMERIC(14,2), reserved_amount NUMERIC(14,2)
      ,
  status VARCHAR(24), validity VARCHAR(12) DEFAULT 'DAY',
  idempotency_key VARCHAR(64) UNIQUE
trades: id UUID PK, order_id UUID FK, sequence INT,
  quantity INT (>0), price NUMERIC(14,2) (>0),
  UNIQUE(order_id, sequence)
trade_charges: id UUID PK, trade_id UUID UNIQUE FK,
  brokerage, stt, exchange_fee, sebi_fee,
  stamp_duty, gst, total NUMERIC(14,2) (>=0)
positions: id UUID PK, UNIQUE(account_id, instrument_id, product)
      ,
  quantity INT (signed), average_price NUMERIC(14,2) (>=0),
  realized_pnl NUMERIC(14,2)
cash_ledger: id UUID PK, account_id UUID FK,
  entry_type IN ('INITIAL_CAPITAL','TRADE_DEBIT',
    'TRADE_CREDIT','CHARGE','ADJUSTMENT','RESET'),
  amount NUMERIC(14,2), balance_after NUMERIC(14,2) (>=0),
  idempotency_key VARCHAR(64) UNIQUE
portfolio_safepoints: id SERIAL PK, account_id UUID FK,
  name VARCHAR(100), snapshot_data JSON
```

Positioning Statement. Quantx is presented as an empirically verified software-engineering proof-of-concept for a boundary-enforced, depth-walking NSE equity paper-trading and state-recovery workspace whose internal accounting and disorder-resilience invariants are audited on scripted scenarios, not as a replica of live exchange queue execution, a human-subject educational intervention, a brokerage platform, or a financial advisory service.

References

[1] L. Harris, Trading and Exchanges: Market Microstructure for Practitioners. New York, NY, USA: Oxford Univ. Press, 2003.

[2] J. Hasbrouck, Empirical Market Microstructure: The Institutions, Economics, and Econometrics of Securities Trading. New York, NY, USA: Oxford Univ. Press, 2007.

[3] M. D. Gould, M. A. Porter, S. Williams, M. McDonald, D. J. Fenn, and S. D. Howison, "Limit order books," Quantitative Finance, vol. 13, no. 11, pp. 1709–1742, 2013.

[4] J. Gray and A. Reuter, Transaction Processing: Concepts and Techniques. San Mateo, CA, USA: Morgan Kaufmann, 1993.

[5] M. Kleppmann, Designing Data-Intensive Applications. Sebastopol, CA, USA: O'Reilly Media, 2017.

[6] R. Almgren and N. Chriss, "Optimal execution of portfolio transactions," Journal of Risk, vol. 3, no. 2, pp. 5–39, 2001.

[7] A. S. Kyle, "Continuous auctions and insider trading," Econometrica, vol. 53, no. 6, pp. 1315–1335, 1985.

[8] L. R. Glosten and P. R. Milgrom, "Bid, ask and transaction prices in a specialist market with heterogeneously informed traders," Journal of Financial Economics, vol. 14, no. 1, pp. 71–100, 1985.

[9] R. Cont, S. Stoikov, and R. Talreja, "A stochastic model for order book dynamics," Operations Research, vol. 58, no. 3, pp. 549–563, 2010.

[10] B. M. Barber and T. Odean, "Trading is hazardous to your wealth: The common stock investment performance of individual investors," The Journal of Finance, vol. 55, no. 2, pp. 773–806, 2000.

[11] ——, "Online investors: Do the slow die first?" The Review of Financial Studies, vol. 15, no. 2, pp. 455–488, 2002.

[12] B. M. Barber, Y.-T. Lee, Y.-J. Liu, and T. Odean, "Just how much do individual investors lose by trading?" The Review of Financial Studies, vol. 22, no. 2, pp. 609–632, 2009.

[13] T. Akidau, R. Bradshaw, C. Chambers, S. Chernyak, R. J. Fernández-Moctezuma, R. Lax, S. McVeety, D. Mills, F. Perry, E. Schmidt, and S. Whittle, "The dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing," Proceedings of the VLDB Endowment, vol. 8, no. 12, pp. 1792–1803, 2015.

[14] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," Communications of the ACM, vol. 21, no. 7, pp. 558–565, 1978.

[15] J. Hasbrouck and G. Saar, "Low-latency trading," Journal of Financial Markets, vol. 16, no. 4, pp. 646–679, 2013.

[16] Department of Economic and Policy Analysis (DEPA), "Analysis of profit and loss of individual traders dealing in equity F&O segment," Securities and Exchange Board of India (SEBI), Mumbai, India, Tech. Rep., Jan. 2023.

[17] P. Helland, "Idempotence is not a medical condition," Communications of the ACM, vol. 55, no. 5, pp. 56–65, 2012.

[18] J.-P. Bouchaud, J. D. Farmer, and F. Lillo, "How markets slowly digest changes in supply and demand," in Handbook of Financial Markets: Dynamics and Evolution, T. Hens and K. R. Schenk-Hoppé, Eds. Amsterdam, The Netherlands: North-Holland, 2009, pp. 57–160.

[19] D. A. Kolb, Experiential Learning: Experience as the Source of Learning and Development. Englewood Cliffs, NJ, USA: Prentice-Hall, 1984.

[20] D. H. Bailey, J. M. Borwein, M. López de Prado, and Q. J. Zhu, "Pseudo-mathematics and financial charlatanism: The effects of backtest overfitting on out-of-sample performance," Notices of the American Mathematical Society, vol. 61, no. 5, pp. 458–471, 2014.

[21] R. D. Peng, "Reproducible research in computational science," Science, vol. 334, no. 6060, pp. 1226–1227, 2011.

[22] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, Experimentation in Software Engineering. Berlin, Germany: Springer, 2012.
[23] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, USA: Addison-Wesley, 2002.
[24] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Univ. of California, Irvine, Irvine, CA, USA, 2000.
[25] I. Fette and A. Melnikov, "The WebSocket protocol," RFC 6455, Internet Engineering Task Force, 2011.
[26] A. Biryukov, D. Dinu, and D. Khovratovich, "Argon2: New generation of memory-hard functions for password hashing and other applications," in Proc. IEEE Eur. Symp. Security and Privacy (EuroS&P), 2016, pp. 292–302.
[27] M. Jones, J. Bradley, and N. Sakimura, "JSON web token (JWT)," RFC 7519, Internet Engineering Task Force, 2015.
[28] OWASP Foundation, "Forgot password cheat sheet," OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Forgot_Password_Cheat_Sheet.html, accessed: Oct. 22, 2026.